

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«АЛТАЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»  
Институт математики и информационных технологий  
Кафедра теоретической кибернетики и прикладной математики

**Разработка мобильной части информационной системы управления  
мероприятиями**  
выпускная квалификационная работа

Выполнил:  
студент группы 4.201-2,  
Золотухина Анастасия Максимовна

\_\_\_\_\_  
(подпись)

Научный руководитель:  
к.ф.-м.н., доцент  
Журавлёва Вера Владимировна

\_\_\_\_\_  
(подпись)

Работа защищена:  
«\_\_» \_\_\_\_\_ 2026 г.

Оценка: \_\_\_\_\_

Председатель ГЭК:  
д.т.н., профессор  
Фамилия Имя Отчество

\_\_\_\_\_  
(подпись)

Допустить к защите:  
Зав. кафедрой, к.ф.-м.н., доцент  
Понькина Елена Владимировна

\_\_\_\_\_  
(подпись)

«\_\_» \_\_\_\_\_ 2026 г.

Барнаул 2026

## РЕФЕРАТ

Тема выпускной квалификационной работы: «Разработка мобильной части информационной системы управления мероприятиями».

Цель работы – разработка мобильной части информационной системы управления мероприятиями.

Объект исследования – методология и практика разработки клиент-серверных приложений.

Предмет исследования – мобильная разработка на языке Kotlin.

В ходе выполнения работы были решены следующие задачи: изучены основные методы и технологии мобильной разработки, выбраны архитектурные принципы и паттерны проектирования, выбраны программные средства разработки, разработан доменный слой, реализовано взаимодействие с backend-частью, разработан пользовательский интерфейс, разработана система интеллектуального поиска и персонализированных рекомендаций.

Итогом исследования стала клиентская часть системы «Events App», которая дает пользователям возможность проходить авторизацию, создавать и регистрировать события, видеть персонализированные предложения на основе алгоритма TF-IDF и передавать информацию на сервер. Приложение реализует функциональность интеллектуального поиска мероприятий с поддержкой морфологического анализа текста.

Ключевые слова: разработка мобильных приложений, Kotlin, Android-разработка, фреймворк Jetpack Compose, паттерн MVVM, библиотека Hilt, система управления событиями, рекомендательная система, архитектура клиент-сервер, интерфейс приложения, интеграция с backend-частью, алгоритм TF-IDF, текстовый поиск, Coroutines, Material Design 3, DataStore.

## СОДЕРЖАНИЕ

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                             | 4  |
| 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ МОБИЛЬНОЙ РАЗРАБОТКИ .....                         | 8  |
| 1.1. Концепция мобильного приложения .....                                 | 8  |
| 1.2. Обзор платформы Android и выбор языка Kotlin .....                    | 13 |
| 1.3. Архитектурные принципы и паттерны в мобильной разработке . . . .      | 16 |
| 1.4. Методы создания UI и работы с данными в Android .....                 | 18 |
| 1.5. Алгоритм текстового поиска .....                                      | 23 |
| 1.6. Алгоритм рекомендаций .....                                           | 24 |
| 2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ<br>МЕРОПРИЯТИЯМИ ..... | 27 |
| 2.1. Анализ требований к системе .....                                     | 27 |
| 2.2. Архитектура системы и моделирование компонентов .....                 | 27 |
| 2.3. Моделирование данных на мобильной стороне .....                       | 27 |
| 3. РАЗРАБОТКА МОБИЛЬНОЙ-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ<br>МЕРОПРИЯТИЯМИ .....    | 28 |
| 3.1. Разработка доменного слоя .....                                       | 28 |
| 3.2. Взаимодействие с backend-частью .....                                 | 28 |
| 3.3. Проектирование пользовательского интерфейса .....                     | 28 |
| 3.4. Интеграция рекомендательной системы .....                             | 28 |
| 3.5. Тестирование и оптимизация приложения .....                           | 28 |
| ЗАКЛЮЧЕНИЕ .....                                                           | 29 |
| БИБЛИОГРАФИЧЕСКИЙ СПИСОК .....                                             | 30 |
| ПРИЛОЖЕНИЕ .....                                                           | 34 |

## **ВВЕДЕНИЕ**

**Актуальность исследования.** Мероприятия различного вида проводятся ежедневно в миллионах организаций по всему миру. Организаторам приходится решать множество задач: от выбора подходящего места и формата до распространения анонсов в мессенджерах и социальных сетях. В крупных компаниях, где события организуются регулярно, и организаторы, и участники сталкиваются с проблемами координации: ручное ведение списков регистраций в таблицах или отслеживание изменений в расписании приводят к ошибкам, потере времени и снижению эффективности работы.

Для решения этих проблем предлагается разработать информационную систему управления мероприятиями. Система будет встроена в существующую экосистему организации и позволит контролировать весь процесс в одном месте. Она обеспечит централизованный контроль всех этапов: от создания события с указанием даты, описания и места до управления регистрацией участников и анализа вовлеченности. Мобильная часть предоставит удобный доступ к основным функциям: просмотр списка событий, аутентификация пользователей и синхронизация данных с сервером.

Актуальность темы обусловлена растущей цифровизацией всех отраслей экономики, что влечёт за собой увеличение потребности рынка в новых цифровых решениях для привычных и рутинных задач.

**Степень разработанности проблемы.** В настоящее время существует ряд систем управления мероприятиями: Eventbrite, Google Calendar или корпоративные инструменты такие как Microsoft Teams. Они решают задачи планирования и регистрации, однако зачастую ориентированы на крупные

события и не учитывают специфику внутренних корпоративных процессов. Например, персонализированные рекомендации и полноценная интеграция с мобильными устройствами на базе Android. Автоматизация мобильной части для таких систем остается недостаточно проработанной, особенно в контексте использования Kotlin и современных подходов к разработке. Это создает пробел в доступных решениях для средних организаций.

Разработка предлагаемой информационной системы требует применения современных подходов и технологий: использование паттернов проектирования, фреймворков, технологий тестирования и баз данных.

**Целью** данной выпускной квалификационной работы является разработка мобильной части информационной системы управления мероприятиями.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Изучить основные методы и технологии мобильной разработки.
2. Выбрать архитектурные принципы и паттерны проектирования.
3. Выбрать программные средства разработки.
4. Разработать доменный слой.
5. Разработать взаимодействие с backend-частью.
6. Разработать пользовательский интерфейс.
7. Разработать систему интеллектуального поиска и рекомендаций.

**Объектом исследования** данной работы является методология и практика разработки клиент-серверных приложений.

**Предмет исследования** – мобильная разработка на языке Kotlin.

**Теоретическая и методологическая основа исследования** включает работы отечественных и зарубежных практиков и теоретиков в области мобильной разработки: документацию Android и Kotlin, а также исследования по паттернам проектирования (MVVM) и алгоритмам обработки текста (TF-IDF). Основу составляют технологии Jetpack Compose, Hilt, а также Retrofit для интеграции с backend. Для управления состоянием приложения и асинхронной загрузки данных применяется Coroutines совместно с Material UI.

**Методы исследования:** анализ научной литературы, системный анализ, сравнение технологий мобильной разработки, моделирование архитектуры, программирование на Kotlin, тестирование интерфейса и интеграции.

**Отличительными особенностями разработки** является интеграция системы интеллектуального поиска на основе алгоритма TF-IDF. Данный подход позволяет разбивать поисковые запросы на составляющие компоненты и находить наиболее близкие по тематике мероприятия даже при неполном совпадении, что обеспечит более гибкий поиск по сравнению с традиционными методами. На основе этой же технологии реализована система персонализированных рекомендаций, которая учитывает предпочтения пользователей. Также применяется Jetpack Compose для создания интерфейса, что обеспечивает более плавную и быструю работу по сравнению с традиционными подходами на основе XML. Это повышает удобство для корпоративных пользователей и снижает нагрузку на устройство.

**Практическая значимость** разработки мобильной части системы управления мероприятиями заключается в том, чтобы предоставить удобный и визуально привлекательный интерфейс для работы пользователей. Благодаря системе организация сможет уменьшить время на координацию, повысить

интерес сотрудников с помощью персонализированных рекомендаций и снизить количество ошибок при регистрации. В результате это улучшит внутренние процессы и повысит общую эффективность.

**Структура работы** обусловлена последовательностью решения поставленных задач и состоит из введения, трех глав, заключения, библиографического списка и приложений. Общий объем работы составляет + страниц, работа содержит + рисунков, + таблиц и 1 приложение.

В первой главе рассматриваются теоретические основы мобильной разработки: концепция мобильного приложения, обзор платформы Android и выбор языка Kotlin, инструменты разработки, архитектурные принципы и паттерны, методы создания интерфейса и анализ альтернативных подходов.

Вторая глава посвящена проектированию информационной системы управления мероприятиями: анализ требований, архитектура и моделирование компонентов, моделирование данных на мобильной стороне и проектирование рекомендательной системы.(демо)

В третьей главе представлена разработка мобильной части системы: доменный слой, взаимодействие с backend, проектирование пользовательского интерфейса, интеграция рекомендательной системы, тестирование и оптимизация приложения.

Таким образом, проект представляет собой не только создание мобильного приложения, а комплексное решение для оптимизации корпоративных процессов. Оно интегрирует современные технологии разработки, поисковые и рекомендательные механизмы, заменяя рутинные операции эффективным и удобным инструментом для пользователей.

# 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ МОБИЛЬНОЙ РАЗРАБОТКИ

## 1.1. Концепция мобильного приложения

Мобильные приложения стали неотъемлемой частью повседневной жизни, изменив способ взаимодействия людей с цифровыми технологиями. Согласно данным отчета Digital 2025, пользователи проводят в среднем 5,1 часа в день за мобильными устройствами, из которых 88% времени приходится на приложения. В бизнес-среде такие приложения служат инструментом маркетинга и вовлечения аудитории: компании персонализируют коммуникации через push-уведомления и интеграцию с социальными сетями, что повышает вовлеченность пользователей и оптимизирует рабочие процессы.

Приложение для мобильных устройств – это специально созданный программный продукт, предназначенный для функционирования на портативной технике: смартфонах, планшетах и других устройствах схожего типа. Такое ПО является автономной программой, адаптированной под особенности той или иной операционной системы, что гарантирует эффективное применение ресурсов устройства. В отличие от приложений для ПК, мобильные версии рассчитаны на переносимость, учитывают условия эксплуатации и обеспечивают комфортный интерфейс взаимодействия с пользователем. По технологии разработки мобильные приложения подразделяются на три основных типа.

**Нативные** приложения разрабатываются для определенной операционной системы (Android, iOS и других) с использованием специализированных инструментов и языков программирования данной



платформы. Такие приложения характеризуются высокой скоростью работы, полноценным доступом к аппаратным возможностям устройства, эффективным расходом ресурсов аккумулятора и оперативной памяти, а также способностью работать в автономном режиме без подключения к сети. Основным недостатком является необходимость создания отдельного программного кода для каждой платформы, что значительно увеличивает время и финансовые затраты на разработку.

**Веб-приложения** представляют собой адаптированные версии интернет-сайтов, доступ к которым осуществляется через браузер мобильного устройства. Данный тип приложений не требует установки и работает на любом устройстве, оснащённом браузером. К преимуществам относятся универсальность применения на различных платформах и упрощённый процесс обновления. Недостатками являются ограниченные возможности взаимодействия с аппаратными компонентами устройства, необходимость постоянного интернет-соединения и сниженная производительность в сравнении с нативным.

**Гибридные** приложения объединяют преимущества нативных и веб-приложений. В их основе лежат веб-технологии (HTML, CSS, JavaScript), которые интегрируются в специальную нативную оболочку. Такой подход позволяет размещать приложения в цифровых магазинах Google Play и App Store, применять единую кодовую базу для различных платформ и сокращать расходы на разработку. К недостаткам можно отнести меньшую скорость работы по сравнению с нативными приложениями.

| Факторы                                    | Нативный      | Гибридный                     | Web             |
|--------------------------------------------|---------------|-------------------------------|-----------------|
| Цена поддержки различных платформ и версий | Высокая       | Средняя                       | Низкая          |
| Доступ к возможностям устройства           | Полный        | Полный                        | Частичный       |
| Пользовательский интерфейс                 | Полноценный   | Полноценный                   | С ограничениями |
| Быстродействие                             | Очень высокое | Очень высокое                 | Высокое         |
| Обновление версий клиентского приложения   | Требуется     | Требуется                     | Не требуется    |
| Простота публикации/распространения        | Средняя       | Средняя                       | Высокая         |
| Цикл согласования                          | Обязательный  | Требуется в некоторых случаях | Не требуется    |
| Монетизация через супермаркет приложений   | Доступна      | Доступна                      | Недоступна      |

Рис. 1. Сравнительная характеристика типов мобильных приложений

**Android** представляет собой мобильную операционную систему, созданную на основе модифицированного ядра Linux, разработанную компанией Google. Являясь открытой платформой с доступным исходным кодом, Android предоставляет производителям устройств возможность свободно адаптировать систему под свои требования, внедряя собственные графические оболочки. Данная особенность обеспечивает широкое распространение системы: она поддерживает устройства различных ценовых категорий, от бюджетных до премиум моделей, и предлагает обширную экосистему приложений через магазин Google Play. Однако открытость платформы создает проблемы фрагментации системы и повышенные риски в области информационной безопасности.

**iOS** является операционной системой для мобильных устройств, разработанной компанией Apple. В отличие от Android, iOS представляет собой

закрытую проприетарную платформу, функционирующую исключительно на устройствах Apple. Данный подход обеспечивает высокий уровень интеграции аппаратных и программных компонентов, строгий контроль качества приложений через магазин App Store и повышенную информационную безопасность. Система отличается плавностью пользовательского интерфейса и оптимизацией энергопотребления. К недостаткам относятся высокая стоимость устройств и ограниченные возможности настройки системы.

**HarmonyOS** представляет собой операционную систему, созданную компанией Huawei в качестве альтернативы Android. Это универсальное решение предназначено для устройств Huawei и ориентировано на технологию распределенных вычислений, обеспечивающую бесшовное взаимодействие между различными устройствами. Система построена на архитектуре микроядра, поддерживает запуск приложений Android и предлагает собственные сервисы HMS. К преимуществам относятся высокий уровень безопасности и расширенная поддержка технологий, объединяющих устройства в единую систему. Недостатками являются ограниченная доступность приложений за пределами азиатского региона и зависимость от устройств производства Huawei.

| Параметр                   | Android     | iOS                              | HarmonyOS                       |
|----------------------------|-------------|----------------------------------|---------------------------------|
| Тип платформы              | Открытая    | Закрытая<br>проприетарная        | Закрытая с<br>открытым<br>кодом |
| Доля рынка (2025)          | 70%         | 25-30%                           | 4-5%                            |
| Производительность         | Высокая     | Очень высокая                    | Очень высокая                   |
| Безопасность               | Средняя     | Высокая                          | Высокая (Star<br>Shield)        |
| Интеграция<br>устройств    | Средняя     | Высокая<br>(AirDrop,<br>Handoff) | Очень высокая<br>(IoT)          |
| Экосистема<br>приложений   | Google Play | App Store                        | AppGallery                      |
| Фрагментация               | Высокая     | Низкая                           | Низкая                          |
| География<br>использования | Глобальная  | Глобальная                       | Преимущественно<br>Азия         |

Таблица 1. Сравнительная характеристика мобильных операционных систем

В данной работе была выбрана разработка нативного мобильного приложения для платформы Android на языке Kotlin. Выбор данного подхода связан с необходимостью обеспечить высокую производительность, полный доступ к API платформе и создать пользовательский интерфейс по стандартам Material Design. Система управления мероприятиями будет решать задачи координации процессов, интегрируясь в корпоративную среду и снижая вероятность ошибок при создании событий.

## 1.2. Обзор платформы Android и выбор языка Kotlin

Android предоставляет разработчикам обширный набор инструментов и компонентов для создания приложений. В данном разделе рассматриваются основные компоненты архитектуры Android и преимущества использования языка Kotlin для разработки приложений на этой платформе.

Архитектура операционной системы Android реализована в виде иерархии уровней, каждый из которых отвечает за выполнение специализированных задач, а их взаимодействие обеспечивает корректное функционирование приложения.

Нижний уровень архитектуры составляет ядро Linux. Оно управляет основными ресурсами устройства: памятью, процессорными потоками, сетевыми подключениями и безопасностью системы. Ядро Linux позволяет Android работать на различных мобильных устройствах.

Над ядром находится слой аппаратной абстракции (HAL). Этот слой служит промежуточным звеном между операционной системой и конкретным оборудованием устройства. Благодаря HAL разработчики Android могут создавать приложения без необходимости учитывать конкретные особенности процессора или других компонентов разных устройств.

Среда исполнения Android Runtime (ART) отвечает за запуск приложений. Она преобразует код приложения в машинные инструкции, которые может выполнить процессор. ART использует технику JIT-компиляции, которая во время работы приложения определяет, какие части кода используются часто, и оптимизирует их работу.

На самом верхнем уровне находится фреймворк приложений (Android Framework). Это набор готовых компонентов и инструментов, которые

разработчик использует при создании приложения. Фреймворк предоставляет: системы управления жизненным циклом приложения (Activity, Fragment), средства для работы с базами данных, инструменты для сетевого взаимодействия, системы управления доступом к функциям устройства и хранилищем данных.

Для разработки приложений под Android используются два основных языка программирования: Java и Kotlin.

Java является первым языком платформы Android, для которого существует обширная база библиотек и инструментов разработки.

Kotlin – это более современный язык, созданный компанией JetBrains. Он имеет более удобный синтаксис и решает ряд проблем, характерных для Java. Kotlin сохраняет полную совместимость с Java, что позволяет использовать оба языка в одном проекте и применять существующие Java-библиотеки.

| Параметр            | Java        | Kotlin                |
|---------------------|-------------|-----------------------|
| Объем кода          | Большой     | Компактный            |
| Null-безопасность   | Ручная      | Автоматическая        |
| Асинхронность       | Сложная     | Простая<br>(корутины) |
| Скорость разработки | Медленнее   | Быстрее               |
| Количество ошибок   | Больше      | Меньше                |
| Поддержка Google    | Стандартная | Приоритетная          |
| Совместимость       | Только Java | Java + Kotlin         |

Таблица 2. Сравнительная характеристика языков Java и Kotlin

**Ключевые особенности.** Kotlin предоставляет несколько важных возможностей для разработки надежных мобильных приложений.

Встроенная система защиты от пустых значений предотвращает возникновение ошибок `NullPointerException` – одной из наиболее частых причин сбоев в приложениях. Система типов Kotlin проверяет обращение к потенциально пустым значениям на этапе компиляции, что исключает их возникновение при выполнении приложения.

Корутины предоставляют удобный способ работы с асинхронным программированием. Они позволяют писать асинхронный код в стиле последовательного выполнения, без использования сложных конструкций обратных вызовов. Это особенно полезно при работе с сетевыми запросами и операциями с базами данных.

Автоматическое приведение типов после проверки условия в Kotlin избавляет разработчика от необходимости преобразований типов, что уменьшает количество повторяющегося кода.

**Выбор языка программирования Kotlin** в качестве основного инструмента разработки обусловлен рядом технических и архитектурных преимуществ. Приложение требует надежной обработки пользовательских данных, и встроенная защита от пустых значений в Kotlin критически важна для стабильности работы. Корутины упрощают реализацию фоновых операций, таких как обмен данными с сервером и работа с локальной базой данных. Компактный синтаксис Kotlin ускоряет разработку и повышает читаемость кода, что облегчает его поддержку и развитие функциональности. Полная совместимость с Java обеспечивает доступ ко всей экосистеме Java-библиотек. Официальная поддержка Kotlin от компании Google гарантируют актуальность и постоянное совершенствование языка.

**IntelliJ IDEA Ultimate Edition** от компании JetBrains была выбрана в качестве интегрированной среды разработки. Поскольку Kotlin и IntelliJ IDEA разработаны одной компанией, среда обеспечивает максимальную поддержку языка на всех уровнях. IntelliJ IDEA предоставляет развитые инструменты для рефакторинга, интеграцию с системами контроля версий, гибкую систему плагинов и поддержку множества языков программирования.

### **1.3. Архитектурные принципы и паттерны в мобильной разработке**

В мобильной разработке используются различные архитектурные паттерны. Каждый паттерн определяет способ разделения кода на логические части и распределения ответственности между компонентами. Правильный выбор архитектуры повышает надёжность приложения, упрощает его развитие и облегчает выявление и исправление ошибок.

**MVC (Model-View-Controller)** один из первых паттернов для разделения кода. Он делит приложение на три части: Model хранит данные, View отображает интерфейс, Controller обрабатывает действия пользователя. Однако в Android Controller часто становится очень большим и содержит слишком много кода. В классической реализации Android компоненты Activity и Fragment выполняют роль Controller, что приводит к их перегруженности логикой. Это затрудняет тестирование и поддержку приложения. Поэтому MVC редко используется в современной разработке Android.

**MVP (Model-View-Presenter)** улучшает MVC. Вместо Controller используется Presenter, который находится между Model и View. Presenter обрабатывает все действия пользователя и управляет отображением данных. View становится простым – он только показывает то, что ему передает Presenter, и не содержит никакой логики. Главное преимущество MVP заключается



в возможности тестирования бизнес-логики без запуска Android-окружения. Это облегчает тестирование, так как Presenter можно проверить отдельно от Android-компонентов. Однако MVP требует написания большого количества кода для связи между компонентами через интерфейсы.

**MVVM (Model-View-ViewModel)** современный паттерн, который используется в Android-разработке как стандарт. В MVVM ViewModel – это компонент, который хранит состояние приложения и управляет данными. View отображает информацию и отправляет действия пользователя в ViewModel. Главным преимуществом MVVM является автоматическое обновление интерфейса. ViewModel также переживает изменения конфигурации устройства, такие как поворот экрана, что предотвращает потерю данных при пересоздании Activity. Когда данные в ViewModel меняются, специальные компоненты (LiveData, StateFlow) автоматически уведомляют View об изменениях, и интерфейс обновляется без дополнительного кода.

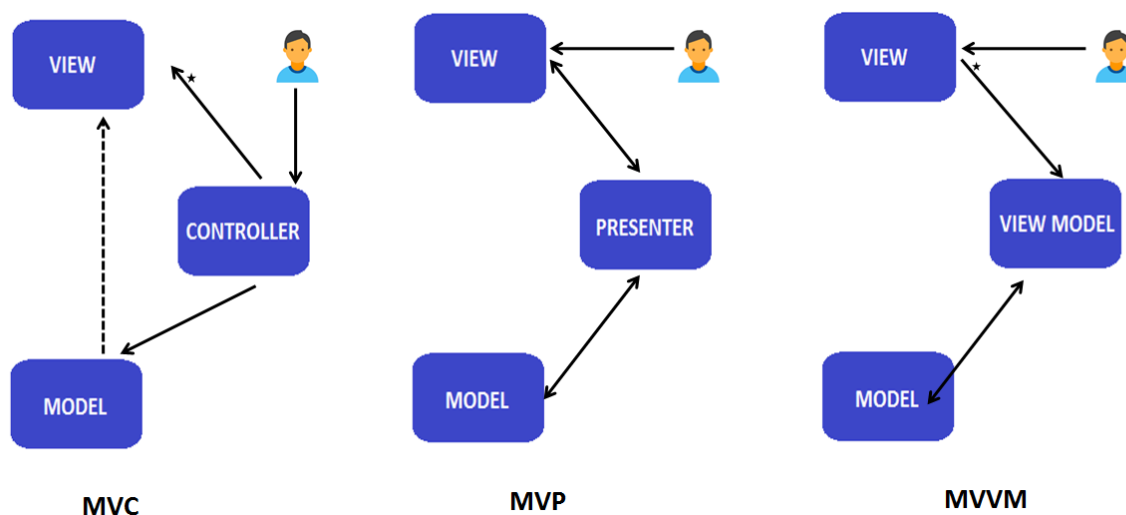


Рис. 2. Архитектурные паттерны

| Параметр                 | MVC        | MVP       | MVVM          |
|--------------------------|------------|-----------|---------------|
| Сложность реализации     | Низкая     | Средняя   | Средняя       |
| Тестируемость            | Низкая     | Высокая   | Высокая       |
| Объем кода               | Небольшой  | Большой   | Средний       |
| Зависимость от платформы | Высокая    | Средняя   | Низкая        |
| Поддержка реактивности   | Нет        | Частичная | Полная        |
| Актуальность для Android | Устаревший | Частично  | Рекомендуется |

Таблица 3. Сравнительная характеристика архитектурных паттернов

Для разработки приложения система управления мероприятиями использует архитектуру MVVM с многоуровневым разделением на слои, рекомендованную Google. Такое разделение обеспечивает модульность приложения и упрощает разработку. Каждый слой может тестироваться независимо. При необходимости изменить способ получения данных нужно изменить только Data Layer, без влияния на остальное приложение. Этот подход соответствует современным стандартам разработки Android и обеспечивает надежность и масштабируемость приложения.

#### 1.4. Методы создания UI и работы с данными в Android

Создание пользовательского интерфейса и организация работы с данными представляют собой ключевые аспекты разработки Android-приложений. В данном разделе рассматриваются современные подходы к

построению UI и методы управления данными, применяемые в мобильных приложениях.

Jetpack Compose представляет собой современный декларативный фреймворк для создания пользовательского интерфейса Android-приложений. В отличие от императивного подхода, где разработчик описывает последовательность действий для построения UI, декларативная модель предполагает описание желаемого состояния интерфейса. Разработчик указывает, как должен выглядеть экран в зависимости от текущих данных, а фреймворк автоматически обновляет отображение при их изменении. Compose-функции являются обычными Kotlin-функциями с аннотацией Composable, что обеспечивает полную интеграцию с языком программирования и его возможностями. Преимущества данного подхода включают значительное сокращение объема кода за счет устранения XML-файлов, улучшенную производительность благодаря интеллектуальной перерисовке только измененных компонентов, упрощенное создание анимаций и встроенную поддержку стандартов Material Design 3.

Material Design представляет собой систему дизайна, разработанную компанией Google, которая определяет принципы создания визуально согласованных пользовательских интерфейсов на различных платформах. Jetpack Compose включает полную поддержку Material Design 3 через библиотеку Material3, предоставляющую готовые компоненты интерфейса. Использование единой системы дизайна обеспечивает согласованный внешний вид приложения и соответствие ожиданиям пользователей Android-устройств.

Для реализации архитектуры MVVM компания Google предоставляет набор библиотек Jetpack Architecture Components. ViewModel представляет

собой компонент для хранения и управления данными пользовательского интерфейса с учетом жизненного цикла Android-компонентов. Ключевая особенность ViewModel заключается в сохранении данных при изменениях конфигурации устройства. Когда происходит поворот экрана, Android по умолчанию пересоздает Activity, что приводит к потере всех данных, хранящихся в памяти. ViewModel сохраняет состояние приложения, что избавляет от необходимости повторной загрузки информации.

StateFlow представляет собой инструмент для работы с потоками данных в реактивном стиле. Данный механизм всегда содержит текущее значение и автоматически уведомляет всех подписчиков об изменениях. StateFlow обеспечивает простую интеграцию с Jetpack Compose и корутинами Kotlin. Когда данные в StateFlow изменяются, все Composable-функции, которые используют эти данные, автоматически перерисовываются. Такой подход реализует принцип реактивного программирования, где интерфейс является функцией от состояния данных. Разработчику не требуется вручную обновлять элементы интерфейса – система выполняет это автоматически.

DataStore представляет собой современное решение для локального хранения данных, разработанное на смену устаревшему SharedPreferences. Библиотека предоставляет два варианта работы: Preferences DataStore для хранения пар ключ-значение и Proto DataStore для типизированных объектов. Основные преимущества включают типобезопасность, асинхронный API на основе корутин и Flow, а также защиту от ошибок при одновременном доступе из разных потоков. Данное решение подходит для хранения настроек приложения, токенов авторизации, пользовательских предпочтений и другой информации небольшого объема.

Для сериализации данных применяется `kotlinx.serialization` – официальная библиотека сериализации для Kotlin. В отличие от библиотек, использующих рефлексю во время выполнения программы, `kotlinx.serialization` генерирует код сериализации на этапе компиляции. Такой подход обеспечивает несколько преимуществ. Ошибки в структуре данных обнаруживаются на этапе компиляции, а не во время работы приложения. Производительность сериализации выше, так как отсутствует необходимость в использовании рефлексии.

Kotlin Coroutines представляют собой механизм для написания асинхронного кода в последовательном стиле. Традиционные подходы к асинхронному программированию, такие как callback-функции или библиотека RxJava, приводят к созданию сложного и трудночитаемого кода. Корутины позволяют писать асинхронные операции так, как если бы они были синхронными. Функции, помеченные ключевым словом `suspend`, могут приостанавливать свое выполнение без блокировки потока. Это позволяет выполнять длительные операции, такие как сетевые запросы или работа с базой данных, не блокируя главный поток интерфейса. Корутины автоматически управляют переключением между потоками и обрабатывают ошибки через стандартные механизмы `try-catch`.

Hilt представляет собой инструмент для внедрения зависимостей, оптимизированный специально для Android. Внедрение зависимостей – это паттерн проектирования, при котором объекты получают свои зависимости извне, а не создают их самостоятельно. Данный подход упрощает тестирование, так как зависимости можно легко заменить на тестовые версии. Hilt построен на основе библиотеки Dagger, но значительно упрощает ее

использование для Android-разработки. Разработчику достаточно пометить классы соответствующими аннотациями – `HiltViewModel` для `ViewModel` и `Inject` для указания зависимостей. Инструмент автоматически создает необходимые объекты и управляет их жизненным циклом в соответствии с жизненным циклом Android-компонентов.

Архитектура работы с данными обычно организована в виде нескольких слоев с четким разделением ответственности. `Data Sources` предоставляют данные из различных источников: локальной базы данных, удаленного сервера, кеша. `Repository` абстрагирует источники данных и предоставляет единый интерфейс для их получения. `Use Cases` инкапсулируют бизнес-логику отдельных операций. `ViewModel` использует `Use Cases` для получения данных и управляет состоянием экрана. Такое разделение обеспечивает независимость слоев друг от друга. Изменение способа получения данных требует модификации только слоя `Repository`, без влияния на остальную систему.

Таким образом, современные инструменты Android-разработки предоставляют комплексное решение для создания приложений. `Jetpack Compose` устраняет недостатки XML-подхода и обеспечивает декларативную модель построения интерфейса. `Material Design` предоставляет готовые компоненты с согласованным дизайном. `ViewModel` и `StateFlow` реализуют паттерн `MVVM` с реактивным обновлением интерфейса. `DataStore` обеспечивает надежное локальное хранение данных. Корутинны упрощают асинхронное программирование. `Hilt` автоматизирует управление зависимостями. Все технологии имеют официальную поддержку Google, активно развиваются и составляют основу современной Android-разработки.

## 1.5. Алгоритм текстового поиска

Поиск по текстовым данным является одной из ключевых функций информационных систем, работающих с контентом. При реализации поиска по словам среди мероприятий возникает необходимость учитывать морфологические особенности русского языка: одно и то же слово может встречаться в разных числовых, временных и падежных формах. Для решения этой проблемы существует два метода нормализации текста – лемматизация и стемминг StemmingLemmatization.

Лемматизация – метод приведения слова к его словарной форме (лемме) с учётом грамматического контекста. Лемматизация опирается на морфологический анализ и словари языка, что обеспечивает высокую точность нормализации. Например, слова «мероприятия», «мероприятие», «мероприятий» будут приведены к форме «мероприятие».

Стемминг – метод приведения слова к его корневой форме путём отсечения окончаний, суффиксов и приставок. В отличие от лемматизации, данный метод не прибегает к морфологическому анализу и словарям, что даёт более быстрый, но менее контекстный результат. Например, слова «мероприятия», «мероприятие», «мероприятий» будут приведены к форме «мероприяти». Метод используется поисковыми движками для более широкого охвата источников информации согласно введенному пользователем ключевому запросу.

Поскольку разрабатываемая система работает с русскоязычными массивами данных, но не требует высокой точности, а также учитывая, что зачастую пользователи используют для поиска 1-2 слова, что показало

исследование Site Search 360 в 2018 году SiteSearch360, в качестве метода нормализации был выбран стемминг.

Алгоритм будет реализован следующим образом. На вход подаётся поисковый запрос пользователя в виде строки, запрос подвергается токенизации, то есть извлечению слов, очищенных от знаков препинания и приведённых к нижнему регистру. Каждый токен подвергается стеммингу, в результате чего формируется множество нормализованных форм поискового запроса.

Параллельно из массива всех доступных мероприятий извлекаются текстовые поля: название, анонс и описание, а также массив тегов. Для каждого мероприятия аналогичным образом выполняется токенизация и стемминг всех слов из указанных полей. Полученные множества стемов хранятся и используются для сопоставления.

На этапе сравнения для каждого мероприятия проверяется наличие пересечения между множеством стемов поискового запроса и множеством стемов мероприятия. Если хотя бы один стем из запроса присутствует в тексте мероприятия, оно включается в результирующую выборку. Итоговый список мероприятий возвращается пользователю в качестве результата поиска.

Такой подход позволяет находить релевантные мероприятия вне зависимости от того, в какой грамматической форме и насколько грамматически верно пользователь вводит поисковый запрос, что существенно повышает удобство и точность поиска.

### **1.6. Алгоритм рекомендаций**

Системы рекомендаций позволяют предлагать пользователю контент, соответствующий его интересам, на основе анализа его предыдущего



поведения. В рамках данной работы будет реализован алгоритм контентной фильтрации, основанный на статистическом методе TF-IDF, который позволяет выявлять ключевые слова, характеризующие интересы пользователя, и на их основе подбирать релевантные мероприятия.

Выбор статистического метода вместо использования нейросетей обоснован требованием к производительности системы, поскольку развёртывание и обучение нейросети потребует значительных финансовых вложений, в то время как TF-IDF не требует отдельных вычислительных мощностей и интеграции с существующим backend.

Рассмотрим алгоритм TF-IDF подробнее.

Term Frequency (TF, частота термина) – характеристика, показывающая, как часто данное слово встречается в массиве слов. Формула расчёта:

$$TF(t, d) = \frac{n_t}{\sum_k n_k},$$

где  $t$  – слово,  $d$  – массив слов,  $n_t$  – количество вхождений слова  $t$  в массив слов,  $\sum_k n_k$  – общее количество слов в массиве.

Inverse Document Frequency (IDF, обратная частота документа) – характеристика, показывающая, насколько редко данное слово встречается во всей коллекции документов. Слова, присутствующие во всех документах, например, предлоги и союзы, получают низкий IDF и фактически исключаются из анализа как незначимые. Формула расчёта:

$$IDF(t, D) = \ln \frac{|D|}{|\{d_i \in D : t \in d_i\}|},$$

где  $t$  – слово,  $D$  – массивы слов,  $|D|$  – количество массивов слов,  $|\{d_i \in D : t \in d_i\}|$  – количество массивов слов, в которых содержится слово  $t$  (когда  $n_t \neq 0$ ).

TF-IDF – итоговая метрика, вычисляемая как произведение TF и IDF. Она позволяет определить слова, которые одновременно часто встречаются в конкретном массиве слов и редко в остальных, то есть слова, наиболее характерные и содержательно значимые для данного текста. Формула расчёта:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \cdot \text{IDF}(t, D)$$

Алгоритм рекомендаций будет реализован следующим образом. На первом этапе из базы данных извлекается список всех мероприятий, которые пользователь посетил ранее. Для каждого такого мероприятия объединяются текстовые поля: название, описание, анонс и теги – в корпус пользователя.

На втором этапе для корпуса вычисляется метрика TF-IDF по всем словам, предварительно подверженным стеммингу. По результатам вычислений отбираются пять слов с наибольшим суммарным значением TF-IDF по всему корпусу пользователя. Эти слова считаются ключевыми для его интересов.

На третьем этапе алгоритм перебирает все доступные актуальные мероприятия, которые пользователь ещё не посещал. Для каждого такого мероприятия проверяется наличие хотя бы одного из пяти ключевых слов в его корпусе. Мероприятия, содержащие ключевые слова, включаются в список рекомендаций и возвращаются пользователю.

Данный подход позволяет формировать персонализированные рекомендации без необходимости сбора данных о других пользователях и без применения коллаборативной фильтрации, что упрощает реализацию и снижает требования к объёму накопленных данных.

## **2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ**

### **2.1. Анализ требований к системе**

### **2.2. Архитектура системы и моделирование компонентов**

### **2.3. Моделирование данных на мобильной стороне**

### **3. РАЗРАБОТКА МОБИЛЬНОЙ-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ**

#### **3.1. Разработка доменного слоя**

#### **3.2. Взаимодействие с backend-частью**

#### **3.3. Проектирование пользовательского интерфейса**

#### **3.4. Интеграция рекомендательной системы**

#### **3.5. Тестирование и оптимизация приложения**

## **ЗАКЛЮЧЕНИЕ**

## БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Артеменко А.В., Бабушкина Н.Е. Ролевая архитектура web-приложений: MVC, MVVM и их влияние на код // Сборник трудов конференции EURASIASCIENCE, 2023. – №1. – С. 82-85.
2. Мясоедова В.А. К вопросу выбора оптимального языка программирования для разработки мобильного приложения // E-SCIO, 2022. – № 11 (74). – С. 119-124.
3. Кузьмин А.В. Мобильные приложения и их роль в современном мире. Типы мобильных приложений. Операционные системы мобильных устройств // Россия молодая, сборник материалов XIII Всероссийской научно-практической конференции молодых ученых с международным участием. – Кемерово, 2021. – С. 31434.1-31434.3.
4. Орлов А.В. Современная разработка Android-приложений и язык программирования Kotlin // Молодежная наука – первый шаг в науку большую, сборник статей III Международной студенческой научно-практической конференции. – Петрозаводск, 2023. – С. 69-73.
5. Петросян Л.Э., Гусев К.В. Разработка мобильных приложений на языке Kotlin: Учебное пособие для вузов. 2-е изд. Лань, 2025. 104 с.
6. Приходько Н.А., Сивченко В.А. Причины выбора Kotlin как основного языка мобильной разработки на Android // Международный журнал гуманитарных и естественных наук, 2023. – № 11-4 (86). – С. 104-107.
7. Первун О.Е., Скрипка В.К. Операционная система Android: история и перспективы // Информационно-компьютерные технологии в экономике, образовании и социальной сфере, 2018. – № 3 (21). – С. 80-87.

8. Сомон., П.И. Волшебство Kotlin : руководство / П. И. Сомон. ; перевод с английского А.Н. Киселева.. – Москва : ДМК Пресс, 2020. – 536 с.
9. Микросервисная архитектура. [Электронный ресурс]. URL: <https://habr.com/ru/companies/raiffeisenbank/articles/346380/> (дата обращения: 27.01.2026).
10. Мобильные операционные системы – обзор ОС | Apptoday [Электронный ресурс]. URL: <https://apptoday.ru/publication/reiting-mobilnykh-operatsionnykh-sistem> (дата обращения: 27.01.2026).
11. Digital 2025, Global Overview Report. [Электронный ресурс]. URL: <https://www.vennapps.com/blog/mobile-app-download-and-usage-statistics-2025> (дата обращения: 05.02.2026).
12. Документация Android. [Электронный ресурс]. URL: <https://developer.android.com/get-started/overview> (дата обращения: 27.01.2026).
13. Документация Flutter. [Электронный ресурс]. URL: <https://flutter.dev/> (дата обращения: 27.01.2026).
14. Документация Hilt. [Электронный ресурс]. URL: <https://dagger.dev/hilt/> (дата обращения: 27.01.2026).
15. Документация Kotlin. [Электронный ресурс]. URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 26.01.2026).
16. Документация Material Design. [Электронный ресурс]. URL: <https://m3.material.io/> (дата обращения: 27.01.2026).
17. Документация React Native. [Электронный ресурс]. URL: <https://reactnative.dev/docs/getting-started> (дата обращения: 27.01.2026).
18. Методология Agile. [Электронный ресурс]. URL: <https://practicum.yandex.ru/blog/metodology-agile/> (дата обращения: 27.01.2026).

19. Рекомендательная система через TF-IDF. [Электронный ресурс]. URL: <https://habr.com/ru/companies/otus/articles/755772/> (дата обращения: 27.01.2026).
20. Рекомендательные системы. [Электронный ресурс]. URL: <https://simulative.ru/blog/rekomendatelnie-sistemi-kak-ml-inzheneri-sozdayut> (дата обращения: 27.01.2026).
21. Функция TF-IDF в Sklearn. [Электронный ресурс]. URL: [https://scikit-learn.org/stable/modules/generated/sklearn.feature\\_extraction.text.TfidfVectorizer.html](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html) (дата обращения: 27.01.2026).
22. Степанов П.В., Приходько Н.А., Запорожских А.И. Разработка мобильных приложений под Android на языке Kotlin : учебное пособие – Москва : МИРЭА - Российский технологический университет, 2025. – 889 с.
23. Stemming, Lemmatization - Which One is Worth Going For? [Электронный ресурс]. URL: <https://towardsdatascience.com/stemming-lemmatization-which-one-is-worth-going-for-77e6ec01ad9c/> (дата обращения: 18.02.2026).
24. Турушев Т.К. Архитектурные паттерны проектирования системы - MVC, MVP, MVVM // Инновационный потенциал развития науки в современном мире, достижения и инновации, сборник научных статей по материалам VI Международной научно-практической конференции. – Уфа, 2021. – С. 16-21.
25. FastAPI. [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 27.01.2026).
26. A Quick Introduction to Bag of Words and TF-IDF. [Электронный ресурс]. URL: <https://towardsdatascience.com/a-quick-introduction-to-bag-of-words-and-tf-idf-fbd3ab84ecbf/> (дата обращения: 18.02.2026).



27. What we learned from 10 million website search tool enquiries. [Электронный ресурс]. URL: <https://www.sitesearch360.com/blog/what-we-learned-from-10-million-website-search-tool-enquiries/> (дата обращения: 18.02.2026).
28. The Destination for Java Developers [Электронный ресурс]. URL: <https://dev.java/> (дата обращения: 21.02.2026).

## **ПРИЛОЖЕНИЕ**

Репозиторий – <https://github.com/Shtskkh/Events.Android>